

ServisAku Partner Backend

Final Delivery Report — Stages 6–9

Admin Backend · Analytics · Testing & QA · Documentation

1. Executive Summary

This report covers Stages 6 through 9 of the ServisAku Partner backend build: the Admin Backend with role-based access control (RBAC), the Analytics reporting layer, an automated test suite, and this documentation pass. It builds directly on Stages 1–5 (Payment Gateway, Media Uploads, Notification Dispatcher, Smart Dispatch, and Real-Time Communication), all previously delivered and live-verified against the team's shared PostgreSQL database.

Across these four stages: 73 new admin endpoints and 11 new analytics endpoints were added (84 total, on top of the roughly 60 endpoints from Stages 1–5); a pre-existing, pre-seeded RBAC schema that no prior code had ever used was wired up end-to-end; a real automated pytest suite was built from nothing, reaching 208 passing tests at 74% statement coverage; and two genuine application bugs were found and fixed during live verification. Every stage's work was pushed to the shared GitHub repository after verification, with a detailed commit history preserved throughout.

The backend remains fully additive to the shared database — zero tables were created, dropped, or redesigned across all four stages. Every new feature maps onto tables that already existed, built by other team members for modules this backend was filling in.

2. Backend Architecture

FastAPI (async) + SQLAlchemy 2.0 (async ORM) + PostgreSQL, structured in four layers: routes (request validation and orchestration only), services (business logic — payment gateways, Smart Dispatch, notifications, RBAC, real-time events), models (thin SQLAlchemy declarative classes), and schemas (Pydantic request/response contracts kept independent of the ORM models, so the external API shape can stay stable even when a table's real column doesn't match it 1:1).

Two integration points use a provider-agnostic pattern uniformly: payments (Billplz/iPay88 behind a common gateway interface) and notifications (push/SMS/email behind a common registry with automatic provider fallback). Real-time events flow through a minimal in-process pub/sub bus (`services/realtime/events.py`) so business logic never imports the Socket.IO layer directly — this avoids a circular import and keeps the real-time transport swappable.

The RBAC layer added in Stage 6 (`services/rbac.py`) sits on top of the existing coarse role check (`partner/consumer/admin` from the JWT) as a second, granular layer: `require_permission("partners.approve")` resolves a user's effective permissions via `user_roles` → `roles` → `role_permissions` → `permissions` — tables that existed and were pre-seeded before this project touched them, but had never been queried by any code.

3. Database Architecture

A single shared PostgreSQL database (`servisakudb`, AWS RDS, PostGIS-enabled), built up incrementally across many team members' modules — 83 tables as of the most recent full schema query, stable since Stage 4. This backend's discipline throughout every stage: query the live schema first, map to what actually exists, never assume or redesign.

- PostGIS geography columns (`partners.home_location`, `consumer_addresses.location`) power Smart Dispatch's proximity search via raw parameterized SQL (`ST_Distance/ST_DWithin`), not ORM-mapped.

- Every enum is Postgres-native (`create_type=False`) — SQLAlchemy never tries to recreate an existing type.
- Identity is centralized in a shared users table, the FK target for every module's user-linked rows.
- `job_dispatches` doubles as both the live offer queue and the permanent dispatch/assignment log — rows are never deleted, only status-transitioned.
- A generated column (`audit_logs.retention_until`) was discovered the hard way in Stage 6 — documented below under Testing Summary.
- No analytics views or materialized views exist in the schema — confirmed before Stage 7 — so every analytics endpoint computes live.

4. API Documentation

Full interactive documentation is served at `/docs` (Swagger UI) and `/redoc`, generated from FastAPI's OpenAPI schema — every endpoint has a description, permission requirement, and example request/response. 25 tag groups organize roughly 144 REST paths (170+ operations counting each HTTP method separately). `API_TESTING_REPORT.md` is the manually-verified companion checklist, tracking pass/fail status and notes per endpoint across every stage.

Stage	Endpoint groups added	Endpoint count
1–3	Consumer, Payments, Uploads, Notification Dispatch	39
4	Smart Dispatch	11
5	Chat (+ Socket.IO events)	4
6	Admin (10 groups: Dashboard, RBAC, Users, Partners, Bookings, Catalog, Coupons, Settlements, Support, Training)	73
7	Analytics	11

5. Authentication Flow

POST `/auth/login` (phone + password) or the OTP verification flow issues a short-lived JWT access token (15 min default) and a longer-lived refresh token (30 days default), each carrying a type claim so one can never be substituted for the other. Every protected route resolves the caller's own scope ID (`partner_id/consumer_id/user_id`) server-side from the token's sub claim — never from a client-supplied parameter — so cross-account access is structurally prevented, not just filtered after the fact. Stage 6 added a second authorization layer (RBAC) on top of this for admin endpoints, detailed in the Backend Architecture section above.

6. Payment Flow

Consumer creates a booking (PENDING_PAYMENT) → POST `/payments/bookings/{id}/bill` creates a gateway checkout bill (Billplz live/self-serve sandbox; iPay88 stubbed pending merchant credentials)

behind a provider-agnostic gateway interface — swapping providers never touches route or business-logic code. The gateway's webhook (POST /payments/billplz/callback) verifies an HMAC signature before trusting any status change, then moves the booking to CONFIRMED and automatically triggers Smart Dispatch. Payments sit in HELD_IN_ESCROW until explicitly released by an admin; refunds follow a full approval workflow (REQUESTED → PENDING_APPROVAL → APPROVED → PROCESSING → COMPLETED/REJECTED/FAILED), every admin decision now logged to the Stage 6 audit trail.

7. Upload Flow

Avatar, KYC document, and job-photo uploads via Cloudinary's free tier, with two paths: a validated server-side upload (simplest) and a signed direct-to-Cloudinary upload for large files on slow connections. Every upload is validated by real magic-byte content sniffing (JPEG/PNG/WEBP signatures), not the client-supplied Content-Type header, which is trivially spoofable — a renamed .txt file is rejected regardless of its declared type. Stage 6 added admin-side KYC document review (verify/reject) on top of the existing partner-side upload flow.

8. Dispatch Flow (Smart Dispatch, Stage 4)

Triggered automatically when a booking's payment is confirmed. A candidate pipeline (services/dispatch/matching.py) finds nearby ACTIVE, available partners via PostGIS within their own service radius, filters by availability slot, skill match, blocked-match exclusion, and daily workload cap, then scores the remainder on a weighted formula (proximity 35%, rating 25%, completion rate 20%, language match 15%, workload 5%). Offers are sequential, not broadcast — exactly one PENDING job_dispatches row exists per booking at a time, with a configurable acceptance timeout, automatic retry to the next-ranked candidate on decline or expiry, and a background sweep worker handling expiries independently of any active request. Manual admin override and full dispatch/assignment history are exposed via the API.

9. Notification Architecture

A central dispatcher (services/notifications/dispatcher.py) always creates the in-app notification row, then best-effort delivers on every other requested channel (push via Firebase, email via a Resend → Brevo → MailerSend fallback chain, SMS via a mock provider pending a real SMS vendor), honoring per-user, per-category channel preferences and logging every delivery attempt — including "no provider configured" — to a dedicated delivery log table. A failing or unconfigured channel never breaks the business action that triggered it (registration, booking confirmation, payment, feedback); this was verified live and one real bug (a background-task database-session race) was found and fixed during Stage 3 development.

10. Socket.IO Architecture (Stage 5)

A JWT-authenticated python-socketio AsyncServer mounted alongside the REST API (uvicorn main:socket_app). Clients join role-scoped rooms (user:{id}, partner:{id}, consumer:{id}, booking:{id}, the last permission-checked per-join) and receive live chat messages, typing indicators, read receipts, partner location updates, presence, heartbeat, and — via the same in-process event bus business logic already emits to — dispatch job offers and booking status updates in real time. Presence and partner GPS location are intentionally not persisted (no dedicated table exists; both are inherently ephemeral). A significant, codebase-wide pre-existing timezone bug (asynccpg silently misinterpreting naive datetimes) and a structlog logging kwarg collision were both found and fixed during this stage's live verification.

11. Analytics Overview (Stage 7)

11 read-only endpoints under `/admin/analytics/*`, gated by the Stage 6 RBAC `reports.read` permission: revenue, bookings, partner performance, consumers, trend metrics (a time-series companion to the Stage 6 dashboard's point-in-time snapshot), conversion funnel, cancellations, dispatch (a thin alias for the existing Stage 4 endpoint — verified byte-for-byte identical, not a duplicated implementation), payments, notifications, and support. No analytics view or materialized view exists in the live schema — confirmed before writing any code — so every number is computed live via aggregate SQL against tables already mapped in earlier stages.

12. Testing Summary (Stage 8)

A real automated pytest suite was built from scratch (none existed before Stage 8): unit tests (password hashing, JWT round-trips, upload MIME sniffing, KYC status mapping — no DB or network), API tests across every route file from every stage, genuine integration tests (a support-ticket lifecycle, a catalog CRUD lifecycle, an RBAC role-assignment lifecycle, and an end-to-end Smart Dispatch flow exercising real candidate ranking and the full start/decline/retry/accept engine path against the live database), and Socket.IO connection tests.

Metric	Result
Tests passing	208 / 208
Tests failing	0
Statement coverage (application code)	74%
Real application bugs found	0 (expected — surface already thoroughly live-verified)
Test bugs found and fixed during development	4 (wrong endpoint paths/shapes, a Decimal-serializes-as-string gotcha)

The gap to the 80% coverage target is concentrated almost entirely in code paths gated behind unconfigured third-party credentials (Billplz, Cloudfire, Firebase, email providers — a gap documented since Stage 1–3, not introduced by this stage) and in `services/realtime/socket_server.py`, whose coverage number is understated because it runs in a separate uvicorn process that pytest-cov cannot instrument — that file was independently and extensively live-verified during Stage 5. Every validation, error-handling, and RBAC code path that doesn't require a live external call is tested.

The suite runs against the real shared development database — no separate test database is provisioned for this project. This is a deliberate, documented tradeoff (`docs/TESTING_GUIDE.md`): it caught two real schema-shape bugs a mocked database would have missed, at the cost of tests not being isolated from concurrent manual/team activity. Every data-creating test uses unique, timestamp-suffixed identifiers and is written to be safe to re-run indefinitely.

13. Security Review

JWT authentication (bcrypt-hashed passwords, separate access/refresh token types), two-layer authorization (coarse role check + Stage 6's granular RBAC, verified to fail closed for a zero-permission role), server-resolved ownership checks throughout (never client-supplied), Pydantic-validated input on

every request body, magic-byte content validation on every file upload, and parameterized SQL everywhere (including the raw-SQL PostGIS and analytics queries). A full audit trail (admin_actions + audit_logs) covers every admin mutation across every stage, not just Stage 6's own new endpoints.

Known gaps, disclosed rather than hidden:

- No rate limiting on any endpoint — recommended before public launch.
- CORS is wide open (ALLOWED_ORIGINS=["*"]) — a development default that must be restricted before production.
- No JWT secret rotation / per-token revocation mechanism.
- The default JWT_SECRET_KEY in config.py is a placeholder — must be overridden in production.

Full detail in docs/SECURITY.md.

14. Deployment Instructions

No production deployment target exists yet — development and verification have run against the shared RDS instance via an SSH tunnel from a local machine. The real ASGI entrypoint for production is uvicorn main:socket_app (not main:app, which drops real-time functionality). A known scaling caveat: the real-time event bus is in-process, so multi-worker/horizontal scaling needs either a Redis-backed Socket.IO message queue or sticky sessions before it can be safely load-balanced. Full checklist — environment variables, TLS termination, background-worker behavior, and recommended next steps — is in docs/DEPLOYMENT.md.

15. Git Commit History (Stages 6–9)

Pushed to origin/main at github.com/Dineshkuppuraj17/servisaku-partner-consumer. Each stage is one feature commit plus a short docs-update commit recording its own hash in docs/today-work/GIT_COMMITS.md — see that file for the complete history including Stages 1–5 and the repository migration.

Commit	Description
c07698f	feat(admin): Stage 6 Admin Backend — RBAC, catalog, partner approval, ops
326d9a2	docs: record Stage 6 commit hash
8e53f32	feat(analytics): Stage 7 Analytics — 11 read-only reporting endpoints
109da90	docs: record Stage 7 commit hash
7085a82	test(qa): Stage 8 Testing & Quality Assurance — pytest suite, 208 tests
bda3615	docs: record Stage 8 commit hash

16. Future Improvements

- Obtain real Billplz, Cloudinary, Firebase, and email provider credentials — the single biggest remaining gap, blocking full end-to-end verification and full test coverage of those integrations.
- Add rate limiting and restrict CORS before any public/production launch.
- Resolve the jobs vs. bookings parallel-table structure (flagged since the original Payment Gateway stage) — payments live on bookings, not this app's own jobs table, and the two remain unconnected.
- Fix the codebase-wide asyncpg naive-datetime timezone bug in the pre-Stage-4 code paths that still use `datetime.utcnow()` instead of `datetime.now(timezone.utc)`.
- Decide and implement a Socket.IO horizontal-scaling strategy (Redis-backed message queue) before deploying more than one worker process.
- Wire CI to run the pytest suite automatically on every pull request.
- Consider a per-role permission audit for the pre-seeded RBAC data — the `READ_ONLY` role currently has zero granted permissions, which may or may not be the intended design.
- Build out the deferred Stage 6/7 partial features once real usage data exists: training-progress write path for partners, subscription/package creation flow, surge-pricing-rule activation logic.

This concludes the Stage 6–9 delivery. All work is committed and pushed to `origin/main`; no further implementation was started pending review and approval of this report.